

本小节内容

逻辑运算符与逻辑表达式

赋值运算符

求字节运算符 sizeof

1 逻辑运算符与逻辑表达式

逻辑运算符!、&&、||依次为逻辑非、逻辑与、逻辑或，这和数学上的与、或、非是一致的。逻辑非的优先级高于算术运算符，逻辑与和逻辑或的优先级低于关系运算符。逻辑表达式的值只有真和假，对应的值为 1 和 0。下例中的代码是计算一年是否为闰年的例子，因为需要重复测试，所以我们用了一个 while 循环。

针对代码中的逻辑非，首先给变量j赋值 10，因为j的值非 0，所以!j的值为 0；然后，由于逻辑非是单目运算符，结合顺序为从右至左，得到!!j的值为 1。也就是说，对 0 取非，得到的值为 1；对非 0 值取非，得到的值为 0。

【例】逻辑运算符的使用。

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i=0,j=1;
    while(scanf("%d",&i))
    {
        if(i%4==0 && i%100!=0 ||i%400==0)
        {
            printf("i is leap year\n");
        }else{
            printf("i is not leap year\n");
        }
    }
    i=!j;
    printf("i 的值=%d\n",i);
    return 0;
}
```

下面我们来看一个短路运算例子：

```
#include <stdio.h>
```

```
int main()
{
    int i=0;
    i&&printf("you can't see me\n");
    return 0;
}
```

逻辑与短路运算是当前面一个表达式为假时，后面的表达式不会得到执行，逻辑或短路运算是当前面一个表达式为真时，后面的表达式不会得到执行。

2 赋值运算符

为了理解有些操作符存在的限制，必须理解左值 (L-value) 和右值 (R-value) 之间的区别。这两个术语多年前由编译器设计者创造并沿用至今，尽管它们的定义与 C 语言并不严格吻合。

左值是那些能够出现在赋值符号左边的东西，右值是那些可以出现在赋值符号右边的东西。例如，

```
a = b + 25;
```

其中，a 是一个左值，因为它标识了一个可以存储结果值的地点；b + 25 是一个右值，因为它指定了一个值。

它们可以互换吗？比如下面这种写法：

```
b + 25 = a;
```

因为每个位置都包含一个值，所以原先用作左值的 a 此时也可以作为右值；然而，b + 25 不能作为左值，因为它并未标识一个特定的位置（并不对应特定的内存空间）。因此，上面这条赋值语句是非法的。

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int a=1,b=2;  
  
b+25=a;  
  
return 0;  
  
}
```

上面的例子执行时会报下面的编译错误：

error: lvalue required as left operand of assignment

接下来介绍复合赋值运算符。

复合赋值运算符操作是一种缩写形式，使用复合赋值运算符能使对变量的赋值操作变得更加简洁。例如，

```
iNum = iNum + 5;
```

对变量 iNum 的赋值进行操作，值为这个变量本身与一个整型常量 5 相加的结果。使用复合语句可以实现同样的操作。例如，上面的语句可以修改为

```
iNum+=5;
```

赋值运算符与复合赋值运算符的区别如下：

- (1) 复合赋值运算符简化了程序，可使程序精炼，提升阅读速度。
- (2) 复合赋值运算符提高了编译效率。

下例说明了加后赋值与乘后赋值的用法。

【例】加后赋值与乘后赋值的用法。

```
#include <stdio.h>  
#include <stdlib.h>  
int main()  
{  
    int iNum,iResult;  
    iNum=10;  
    iResult=3;  
  
    iNum+=5;  
    iResult*=iNum;  
    printf("iNum=%d\n",iNum);  
    printf("iResult=%d\n",iResult);  
}
```

从上面的程序代码可以看到，iNum+=5 代表 iNum 加 5 后再赋值给 iNum，因此 iNum 的最终值为 15，而 iResult 的值等于其自身乘以 iNum 的值，所以最终结果为 45。

3 求字节运算符 sizeof

很多同学会认为 sizeof 是一个函数，这种理解是错误的，实际 sizeof 是一个运算符，不像其他运算符是一个符号，sizeof 是字母组成的，用于求常量或变量所占用的空间大小，请看下例：

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i=0;
```

```
    printf("i size is %d\n",sizeof(i));
```

```
    return 0;
```

```
}
```

运行结果为 i size is 4，可以求得整型变量占用的空间大小是 4 个字节。